I am learning LightGBM I want you to write simplest python script so that I can implement and understand LightGBM Classification and regression .
Use some online dataset that is good for LightGBM.
Write 2 code files
Write the simplest code possible
Print graph
make sure to calculate the accuracy and loss

Python

```python
# classification.py
# Simplest LightGBM classification example using Iris dataset

import lightgbm as lgb
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, log_loss

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# Create LightGBM datasets
train_data = lgb.Dataset(X_train, label=y_train)
test_data = lgb.Dataset(X_test, label=y_test, reference=train_data)

# Parameters
params = {
    'objective': 'multiclass',
    'num_class': 3,
    'metric': 'multi_logloss',
    'verbose': -1
}
```

```python
# Train model
model = lgb.train(params, train_data, num_boost_round=20, valid_sets=[tes

# Predict
y_pred_prob = model.predict(X_test)
y_pred = y_pred_prob.argmax(axis=1)

# Calculate accuracy and loss
accuracy = accuracy_score(y_test, y_pred)
loss = log_loss(y_test, y_pred_prob)
print(f'Accuracy: {accuracy:.4f}')
print(f'Log Loss: {loss:.4f}')

# Plot feature importance
lgb.plot_importance(model)
plt.show()
```

Python

```python
# regression.py
# Simplest LightGBM regression example using California Housing dataset

import lightgbm as lgb
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
housing = fetch_california_housing()
X = housing.data
y = housing.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# Create LightGBM datasets
train_data = lgb.Dataset(X_train, label=y_train)
test_data = lgb.Dataset(X_test, label=y_test, reference=train_data)

# Parameters
params = {
    'objective': 'regression',
    'metric': 'mse',
```

```
    'verbose': -1
}

# Train model
model = lgb.train(params, train_data, num_boost_round=20, valid_sets=[tes

# Predict
y_pred = model.predict(X_test)

# Calculate "accuracy" (R2 score) and loss (MSE)
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
Score (Accuracy): {r2:.4f}')
(Loss): {mse:.4f}')

# Plot feature importance
lgb.plot_importance(model)
plt.show()
```

Add to chat

Answer in very short What is this?

Create LightGBM datasets train_data = lgb.Dataset(X_train, label=y_train) test_data = lgb.Dataset(X_test, label=y_test, reference=train_data)

This creates LightGBM's optimized `Dataset` objects:

- `train_data` : training data with features (`X_train`) and labels (`y_train`).
- `test_data` : validation data; `reference=train_data` links it to training for consistent handling (e.g., categorical features) and enables automatic metric tracking during training.

Parameters params = { 'objective': 'multiclass', 'num_class': 3, 'metric': 'multi_logloss', 'verbose': -1 }

These parameters configure LightGBM for **multiclass classification**:

- `'objective': 'multiclass'` → Tells LightGBM it's a multi-class problem (not binary or regression).
- `'num_class': 3` → Specifies there are 3 classes (required for multiclass; matches Iris dataset).

- `'metric': 'multi_logloss'` → Uses multiclass log loss (cross-entropy) to evaluate model performance.
- `'verbose': -1` → Suppresses training logs/output for cleaner output.

Simple and essential settings for basic multiclass training.

↳ Explain LightGBM hyperparameters tuning

↳ Compare LightGBM vs XGBoost